

TP previo Parcial 2 DDS 2026 - Curso 3K7

Campo	Valor
Materia	Desarrollo de Software (DDS)
Año	2026
Instancia de evaluación	TP previo - Parcial 2
Documento	Enunciado de trabajo práctico
Curso	3K7

Introducción

Adicionalmente cada alumno deberá subir el trabajo realizado a su portafolio.

Desde el 14/6 10 hs hasta el lunes 15/6 a las 23:55 otro grupo deberá realizar una revisión/corrección entre pares (revisión entre grupos) y entregar la devolución grupal.

Estas dos tareas son esenciales y previas al segundo parcial ya que habrá 2 preguntas a desarrollar relativas al tema desarrollado.

Durante una semana el grupo va a desarrollar una aplicación full stack para gestion de tutorías. El sistema debe permitir que estudiantes soliciten turnos, que tutores administren su agenda y que administradores supervisen disponibilidad, estados y conflictos. El objetivo es resolver un flujo completo de reserva, confirmacion y seguimiento, con reglas de horario y permisos reales.

Este práctico integra los contenidos principales del segundo parcial: Express Router, middlewares, manejo centralizado de errores, CORS, autenticación y autorización con JWT, pruebas de API con Jest y Supertest, React con Vite, componentes, estado, efectos, React Router, formularios, Axios, validaciones y manejo de errores de API.

Como el trabajo se resuelve en grupo y con una semana de desarrollo, se espera una solución de complejidad media-alta: reglas de negocio reales, estados, permisos, historial, filtros avanzados, persistencia, datos semilla, pruebas automatizadas y una interfaz navegable que permita operar el sistema completo.

Alcance mínimo de resolución grupal

La entrega debe evidenciar que hubo división de responsabilidades y posterior integración. Como mínimo, el sistema debe incluir:

un módulo de autenticación completo, con registro, login, JWT, roles y persistencia de sesión en **frontend**;

un módulo principal del dominio con listado, filtros, detalle, alta, edición y acciones de estado;

un módulo de administración o resumen que muestre información agregada del dominio;

validaciones de negocio implementadas en servicios del backend, no solamente en formularios;

al menos dos flujos end-to-end funcionando desde la interfaz hasta la persistencia;

pruebas automatizadas que cubran casos exitosos, errores de validación, errores de permisos y **reglas específicas del dominio**;
documentación suficiente para que otro grupo pueda ejecutar, probar y explicar la solución.

No se considera suficiente una entrega que solo tenga pantallas sin backend real, endpoints sin frontend integrado, datos hardcodeados en componentes, o reglas de negocio aplicadas únicamente de forma visual.

Que tienen que construir

Desarrollar una aplicación web full stack para gestion de tutorías. La aplicación debe permitir reservar turnos con tutores, evitar superposiciones y controlar permisos según usuario. No es un CRUD genérico: cada turno debe estar asociado a un tutor, a una franja horaria valida y a un estudiante autenticado.

En el parcial presencial cada estudiante respondera preguntas escritas sobre la solución entregada por su grupo. Deben poder explicar rutas, componentes, servicios, middlewares, validaciones, JWT, permisos, pruebas y errores manejados.

Dominio obligatorio

El sistema trabaja sobre el dominio de gestion de tutorías. El recurso principal que se administra es el Turno: una reserva de atencion entre un estudiante y un tutor, con fecha, horario, tema, modalidad y estado. Cada turno debe estar asociado obligatoriamente a un Tutor, que define especialidad, disponibilidad y estado de actividad.

La regla central del dominio es que un turno solo puede solicitarse, confirmarse o reprogramarse si el tutor existe, está activo, atiende ese día y no tiene otro turno solicitado o confirmado en una franja horaria superpuesta.

Entidades mínimas obligatorias

El sistema debe modelar y persistir al menos 4 entidades. Sí usan archivo JSON, SQLite, Sequelize u otra alternativa, igual deben respetar está separacion lógica:

Entidad	Para qué existe	Campos mínimos	Relaciones principales
usuarios	Representa estudiantes, tutores y administradores que operan el sistema.	id, nombre, email, passwordHash, rol, activo	Un usuario puede solicitar turnos, actuar como tutor y generar historial.
tutores	Representa perfiles de tutor con disponibilidad y especialidad.	id, usuarioId, nombre, email, especialidad, díasDisponibles, activo	Un tutor está vinculado a un usuario y puede tener muchos turnos.
turnos	Representa una reserva de tutoría con fecha, horario y tema.	id, tutorId, estudianteId, fecha, horaInicio, horaFin, tema, modalidad, estado, observaciones	Pertenece a un tutor y a un estudiante solicitante.
historial_turnos	Registra auditoría de cambios sobre turnos.	id, turnoId, usuarioId, accion, fechaHora, valorAnterior, valorNuevo	Pertenece a un turno y al usuario que hizo la acción.

Campos mínimos por entidad

Los campos pueden adaptarse al motor de persistencia elegido, pero la información y las relaciones deben estar presentes.

Usuario

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del usuario.	1, usr-001
nombre	string	Sí	Nombre visible del usuario.	Valen Acosta

email	string	Sí	Credencial de acceso y dato de contacto. Debe ser único.	valen@dds.com
passwordHash	string	Sí	Contraseña almacenada de forma segura o hash simulado documentado.	bcrypt_hash
rol	string	Sí	Define permisos dentro del sistema.	estudiante, tutor, admin
activo	boolean	Sí	Permite bloquear accesos sin borrar el usuario.	true, false

Tutor

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del perfil de tutor.	tut-001
usuarioId	número/string	Sí	Usuario asociado al perfil de tutor.	usr-tutor-1
nombre	string	Sí	Nombre visible del tutor.	Marina López
email	string	Sí	Contacto del tutor.	marina@dds.com
especialidad	string	Sí	Tema principal de tutoría.	backend, frontend, testing, seguridad
díasDisponibles	array/string	Sí	Días en los que puede atender.	["lunes", "miércoles"]
activo	boolean	Sí	Indica si puede recibir nuevos turnos.	true, false

Turno

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del turno.	tur-1001
tutorId	número/string	Sí	Tutor seleccionado. Debe existir en tutores.	tut-001
estudianteId	número/string	Sí	Usuario estudiante que solicita el turno.	usr-001

fecha	date/string	Sí	Día solicitado para la tutoría.	2026-06-18
horaInicio	string	Sí	Inicio de la franja horaria.	10:00
horaFin	string	Sí	Fin de la franja horaria. Debe ser mayor que horaInicio.	10:30
tema	string	Sí	Tema concreto de consulta.	JWT y middlewares
modalidad	string	Sí	Forma de atención.	presencial, virtual
estado	string	Sí	Estado actual del flujo.	solicitado, confirmado, cancelado, realizado
observaciones	string	No	Comentarios del tutor o estudiante.	Traer error de Supertest

HistorialTurno

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del registro de auditoría.	hist-001
turnoId	número/string	Sí	Turno afectado por el cambio.	tur-1001
usuarioId	número/string	Sí	Usuario que realizó la acción.	usr-tutor-1
accion	string	Sí	Operación realizada sobre el turno.	creacion, edicion, confirmación, cancelación, realización
fechaHora	datetime/string	Sí	Momento exacto de la acción.	2026-06-08T11:15:00
valorAnterior	object/string	No	Datos relevantes antes del cambio.	{ "estado": "solicitado" }
valorNuevo	object/string	No	Datos relevantes después del cambio.	{ "estado": "confirmado" }

Reglas funcionales específicas

La aplicación debe resolver estos casos:

1. Registrar usuario e iniciar sesión.
2. Obtener un JWT al iniciar sesión correctamente.
3. Listar turnos con filtros combinables por fecha, estado, tutorId y especialidad.
4. Ver el detalle de un turno desde una ruta dinámica.
5. Crear un turno seleccionando un tutor existente.
6. Editar fecha, horario, tema, modalidad u observaciones según permisos.
7. Cancelar un turno cambiando estado a cancelado.
8. Confirmar o marcar como realizado según rol.
9. Rechazar turnos si horaInicio no es menor que horaFin.
10. Rechazar turnos si el tutor no está activo o si ya tiene un turno solicitado o confirmado en una franja superpuesta.

Complejidad adicional obligatoria

El grupo debe implementar también:

1. **Flujo completo de estados: solicitado → confirmado o cancelado; confirmado → realizado o cancelado.** No permitir modificar turnos realizado salvo observaciones.
2. Búsqueda paginada y ordenable: aceptar page, limit, sortBy y order en el listado de turnos.
3. Vista resumen para administración con turnos por tutor, turnos del día, temas mas solicitados y turnos pendientes de confirmacion.
4. Historial de cambios de cada turno: guardar fecha, usuario, acción y valores modificados cuando se confirma, cancela, realiza, reasigna o edita.
5. Validacion de disponibilidad por día: el tutor debe tener el día de la semana de la fecha solicitada dentro de díasDisponibles.
6. Control de superposición también al editar o reasignar tutor, ignorando el propio turno pero comparando contra los demas.
7. Semilla de datos inicial con al menos 5 tutores, 3 estudiantes, 1 admin y 12 turnos en distintos estados.
8. Mensajes de error distintos para tutor inexistente, tutor inactivo, día no disponible, superposición horaria y falta de permisos.

Aclaraciones para resolver el alcance

Estas aclaraciones responden dudas esperables al momento de implementar:

Cuando un tutor está disponible? Debe estar activo, tener el día de la fecha solicitada dentro de díasDisponibles y no tener otro turno superpuesto.

Que turnos bloquean disponibilidad? Los turnos solicitado y confirmado del mismo tutor y fecha.

Que pasa si dos turnos se tocan justo en el límite horario? Sí uno termina a las 11:00 y otro empieza a las 11:00, no se considera superposición.

Quien puede confirmar o realizar un turno? El tutor asignado puede confirmar o marcar realizado

sus turnos; el administrador puede intervenir sobre cualquier turno.

Que debe mostrar el resumen? Como mínimo: turnos del día, turnos por tutor, temas mas solicitados y turnos pendientes de confirmacion.

Donde se valida disponibilidad y superposición? En el servicio del backend, tanto al crear como al editar o reasignar tutor.

Roles y permisos

Deben existir al menos estos roles:

estudiante: puede crear turnos, ver sus turnos y cancelar sus propios turnos solicitados o confirmados.

tutor: puede ver turnos asignados, confirmar o marcar como realizado.

admin: puede ver todos los turnos, reasignar tutor, confirmar, cancelar o marcar como realizado cualquier turno.

Al menos una ruta de escritura debe devolver:

401 si no se envia JWT.

403 si el usuario está autenticado pero no tiene permiso para la acción.

Backend esperado

Implementar backend con Node.js y Express.

Rutas mínimas:

POST /api/auth/register

POST /api/auth/login

GET /api/tutores

GET /api/turnos?fecha=&estado=&tutorId=&especialidad=

GET /api/turnos/resumen

GET /api/turnos/:id

GET /api/turnos/:id/historial

POST /api/turnos

PUT /api/turnos/:id

PATCH /api/turnos/:id/cancelar

PATCH /api/turnos/:id/confirmar

PATCH /api/turnos/:id/realizar

Estructura minima:

`routes/turnos.routes.js` con `express.Router()`

controlador de turnos

servicio de turnos con regla de disponibilidad y superposición por tutor

middleware de autenticación JWT

middleware de autorización por rol, tutor asignado o estudiante propietario

middleware de validación de entrada

middleware de manejo de errores con firma (`err, req, res, next`)

persistencia en archivo JSON, SQLite, Sequelize u otra solución que conserve datos al reiniciar el

backend

carga de datos semilla para probar el dominio sin cargar todo manualmente

Usar status HTTP coherentes: 200, 201, 400, 401, 403, 404 y 500. Las respuestas de error deben tener JSON claro, por ejemplo { "error": "El tutor no está disponible en esa franja horaria" }.

Frontend esperado

Implementar frontend con React y Vite.

Pantallas mínimas:

1. Login y registro.
2. Listado de turnos con filtros por fecha, estado, tutor y especialidad.
3. Detalle de turno en una ruta como /turnos/:id.
4. Pantalla transaccional de alta/edición de turno, donde se selecciona tutor, fecha, horario, tema y modalidad, y se confirma la operación contra la API.
5. Acciones visibles para cancelar, confirmar o realizar según rol.
6. Panel resumen para administradores.
7. Historial visible dentro del detalle de un turno.
8. Ruta comodín para página no encontrada.

La capa de servicios debe usar Axios:

- **instancia con baseUrl**
- **params para filtros**
- **body en POST, PUT o PATCH**
- **header Authorization: Bearer <token> en acciones protegidas**
- manejo visible de errores de validación, autenticación, autorización y recurso inexistente
- componentes separados para tabla/listado, filtros, formulario, detalle, acciones por rol y resumen **administrativo**
- **estados de carga, vacío, error y éxito para las operaciones principales**

Integración y calidad esperada

La resolución debe mostrar trabajo de equipo y no una acumulación de archivos sueltos. Se espera:

- contexto, hook o mecanismo equivalente para conservar usuario autenticado, token y rol en **frontend**;
- **rutas protegidas en frontend y backend, no solo botones ocultos**;
- validaciones repetidas en frontend para experiencia de usuario y en backend como fuente de **verdad**;
- servicios Axios separados por recurso, sin llamadas HTTP mezcladas dentro de todos los **componentes**;
- contraseñas hasheadas o, si usan usuarios semilla simplificados, aclaracion explicita de la **limitacion en el README**;
- **payload del JWT sin contraseñas ni datos sensibles**;
- **manejo de errores centralizado en backend y mensajes comprensibles en frontend**;

- decision documentada sobre persistencia, estructura de carpetas y división de responsabilidades del grupo.

Testing mínimo

El backend debe incluir pruebas con Jest y Supertest para:

1. Login correcto e inválido.
2. Listado de turnos con y sin filtros.
3. Detalle de turno existente e inexistente.
4. Creacion valida de un turno.
5. Creacion inválida por horario inconsistente.
6. Creacion inválida por superposición del tutor.
7. Acceso sin JWT a una ruta protegida.
8. Acceso con JWT de estudiante a una acción solo permitida para admin o tutor.
9. Edicion inválida que reasigna a un tutor ocupado.
10. Creacion inválida por día no disponible para el tutor.

Las pruebas deben verificar status HTTP y cuerpo JSON relevante.

Documentacion obligatoria

La entrega debe incluir un README.md con:

instrucciones para ejecutar backend y frontend
usuario admin, usuario tutor y usuario estudiante de prueba
listado de endpoints principales
listado de rutas frontend
explicación de disponibilidad y superposición por tutor

explicación de JWT, roles y permisos
comando para ejecutar pruebas
limitaciones conocidas

Preguntas frecuentes y criterios de correccion

Sobre alcance y evaluacion

El TP es condicion de entrega y también sera la base de preguntas abiertas del Parcial 2. La nota del parcial surge del examen, pero una entrega incompleta o no explicable afecta directamente las respuestas escritas.

En el parcial se preguntara sobre el código real entregado por el grupo: rutas, servicios, componentes, validaciones, JWT, permisos, pruebas y decisiones.

Se aceptan soluciones equivalentes si respetan entidades mínimas, reglas del dominio, rutas, pantallas, pruebas y criterios de seguridad. Cualquier cambio importante debe estar justificado en el README.

La persistencia puede ser archivo JSON, SQLite, Sequelize u otra alternativa, siempre que conserve datos al reiniciar y respete las entidades del enunciado.

Debe haber registro/login real desde frontend. Los usuarios semilla son obligatorios para probar rapido, pero no reemplazan el flujo de autenticación.

El historial debe existir en backend y verse desde frontend, al menos en el detalle del recurso principal.

La pantalla transaccional puede ser una sola pantalla para alta/edición o dos pantallas separadas, pero debe confirmar la operación contra la API y mostrar errores.

El resumen administrativo debe estar protegido para roles administrativos.

Filtros, paginacion y ordenamiento deben resolverse en backend. El frontend solo envia params y muestra resultados.

Documentar una limitacion no reemplaza un requisito mínimo. Sirve para explicar decisiones o faltantes menores, no para omitir reglas centrales.

Sobre backend

Deben usar `express.Router()` en archivos separados para el recurso principal.

El middleware de errores con firma (`err, req, res, next`) debe quedar después de las rutas.

Los errores de validación deben responder con status coherente, normalmente 400, y JSON claro, por ejemplo `{ "error": "mensaje" }`.

La autorización debe validar rol y, cuando corresponda, propiedad del recurso. No alcanza con que el JWT exista.

El JWT no debe incluir contraseñas ni datos sensibles.

Las contraseñas deben guardarse hasheadas; si se usa una simplificacion por tiempo, debe estar documentada y no exponer contraseñas en respuestas.

Las reglas de negocio deben estar en servicios del backend, no solamente en controladores ni formularios.

Deben estar protegidas las acciones de escritura y cualquier lectura privada o administrativa.

Sobre frontend

React Router debe incluir ruta comodin *.

Las pantallas de detalle deben leer parametros con useParams.

Pueden usar React Hook Form o formularios controlados con useState, pero las validaciones deben verse en pantalla.

Los errores de API deben mostrarse de forma comprensible cerca de la acción que fallo o en un area visible de la pantalla.

Axios debe estar en una capa de servicios separada. No mezclar llamadas HTTP en todos los componentes.

El JWT puede enviarse con interceptores o agregando headers en servicios, pero siempre como `Authorization: Bearer <token>`.

Proteger una ruta en frontend significa impedir acceso visual/navegable si no hay usuario autenticado o rol suficiente; esto no reemplaza la proteccion del backend.

Sobre testing

Se exigen pruebas de backend con Jest y Supertest. Tests de frontend son opcionales.

Cada test debe validar status HTTP y cuerpo JSON relevante.

Deben probar login correcto, login inválido, acceso sin token y acceso con rol insuficiente.

Deben probar reglas específicas del dominio, no solo endpoints felices.

Los datos de prueba deben ser previsibles. Sí un test modifica datos, deben resetearlos o aislarlos para no afectar otros tests.

Entrega

Subir a Moodle el repositorio o archivo comprimido en el portafolio del alumno y en la entrega grupal. No incluir carpetas `node_modules`. Sí deben incluir `package.json`, código fuente, tests y documentación.